ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN: ĐÁNH GIÁ VÀ KIỂM ĐỊNH CHẤT LƯỢNG PHẦN MỀM

Đề tài: Kiểm thử chức năng và hiệu năng cho hệ thống Quản lý thư viện nhạc online

Giảng viên hướng dẫn: TS. Trịnh Thanh Bình

Nhóm: 4

Thành viên:

1. Lê Anh Quân - 23010580.
2. Trần Thảo Vy - 23010588.

Lớp: CSE703010-1-1-26(COUR01.LT2)

Hà Nội, 2026.

MỤC LỤC

PHÂN CÔNG NHIỆM VỤ

1. TỔNG QUAN VÀ PHẠM VI DỰ ÁN 4
    1.1. Giới thiệu 4
    1.2. Mục tiêu 4
    1.3. Phạm vi và phương pháp tổng hợp 4
    1.4. Bố cục 5
2. KIẾN TRÚC VÀ MÔ HÌNH DỮ LIỆU 6
    2.1. Công nghệ sử dụng 6
    2.2. Luồng xử lý 6
    2.3. Các thực thể 6
3. CHỨC NĂNG VÀ PHÂN QUYỀN 7
    3.1. Tài khoản và phiên đăng nhập 7
    3.2. Các nhóm nghiệp vụ 7
    3.3. Giao diện 7
4. CHIẾN LƯỢC VÀ THIẾT KẾ KIỂM THỬ 8
    4.1. Các cấp độ kiểm thử 8
    4.2. Kỹ thuật thiết kế ca kiểm thử 8
    4.3. Truy vết và cách đọc kết quả 8
5. BẢNG TEST CASE VÀ THỐNG KÊ 9
6. KIỂM THỬ MÃ NGUỒN BẰNG JUNIT 11
    6.1. Nguyên nhân sáu test thất bại 12
    6.2. Ý nghĩa và giới hạn 12
    7.1. Collection Postman 13
    7.2. Bộ kiểm thử Selenium 14
    7.3. Đánh giá kết quả endpoint và giao diện 15
8. KIỂM THỬ HIỆU NĂNG BẰNG JMETER 16
    8.1. Kịch bản và tiêu chí 16
    8.2. Kết quả kiểm thử với 120 người dùng 16
    8.3. Kết quả theo ba mức tải 17
    8.4. Đánh giá kết quả hiệu năng 18

9. CÁC VẤN ĐỀ PHÁT HIỆN TỪ MÃ NGUỒN 19
9.1. Xử lý chưa chính xác mã trạng thái HTTP 20
9.2. Sử dụng phương thức GET để xóa tài khoản 20
9.3. Kiểm tra và liên kết dữ liệu chưa nhất quán 20
9.4. Hạn chế trong tìm kiếm và kiểm tra đường dẫn nội dung 21
10. KHẢ NĂNG TÁI LẬP VÀ KẾ HOẠCH CẢI TIẾN 22
10.1. Cấu hình, quy trình build và tài liệu 22
10.2. Kế hoạch cải tiến theo mức độ ưu tiên 22
10.3. Quy trình kiểm thử xác nhận sau cải tiến 23
11. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 24
TÀI LIỆU THAM KHẢO 25

PHÂN CÔNG NHIỆM VỤ

STT	Thành viên	Mã sinh viên	Nhiệm vụ thực hiện	Mức độ hoàn thành
1	Lê Anh Quân	23010580	Phân tích kiến trúc, chức năng và mã nguồn; thực hiện, xây dựng và thực hiện kiểm thử JUnit và Postman; phân tích lỗi HTTP, validation và phân quyền; rà soát báo cáo.	100%
2	Trần Thảo Vy	23010588	Xây dựng và đối chiếu bảng test case; thực hiện kiểm thử Selenium và JMeter; tổng hợp, phân tích kết quả và minh chứng kiểm thử; tham gia rà soát, hoàn thiện báo cáo.	100%

1. TỔNG QUAN VÀ PHẠM VI DỰ ÁN

1.1. Giới thiệu

Hệ thống đã triển khai các nghiệp vụ quản lý bài hát, nghệ sĩ, thể loại và tài khoản người dùng trên nền Spring Boot. Kiến trúc phân tầng giúp tách giao diện, xử lý nghiệp vụ và truy cập dữ liệu. Bộ kiểm thử bao gồm nhiều cấp độ, từ unit test đến kiểm thử giao diện và hiệu năng.

Kết quả kiểm định ghi nhận 138 test JUnit, gồm 131 test đạt, 6 test không đạt, 0 lỗi và 1 test bị bỏ qua. Postman thực hiện 40 test với 35 test đạt và 5 test không đạt. Selenium ghi nhận 66 test đạt, 4 test bị bỏ qua và 2 test xfail. Ba mức tải JMeter có tỷ lệ lỗi tổng thể từ 0,44% đến 0,46%, thời gian trung bình dưới 8 ms và thông lượng tăng theo số người dùng đồng thời.

1.2. Mục tiêu

Mục tiêu của báo cáo là trình bày tổng quan về hệ thống quản lý thư viện nhạc online, làm rõ cấu trúc dự án, công nghệ sử dụng, mô hình dữ liệu và luồng xử lý của các chức năng chính. Trên cơ sở đó, báo cáo phân tích cách tổ chức kiểm thử ở từng cấp độ, từ kiểm thử đơn vị, kiểm thử tích hợp đến kiểm thử endpoint, giao diện và hiệu năng; đồng thời đánh giá mức độ phù hợp của các ca kiểm thử với yêu cầu và hành vi thực tế của ứng dụng.

Báo cáo cũng hướng đến việc tổng hợp, đối chiếu các kết quả và minh chứng kiểm thử nhằm xác định những chức năng đã đáp ứng yêu cầu, các trường hợp còn lỗi và những nội dung chưa được kiểm chứng. Việc phân tích tập trung vào tính đúng đắn của nghiệp vụ, cơ chế xác thực và phân quyền, khả năng xử lý dữ liệu đầu vào, tính nhất quán của dữ liệu và thời gian phản hồi của hệ thống. Qua đó, báo cáo chỉ ra các sai lệch giữa yêu cầu, mã nguồn, tài liệu và kết quả kiểm thử, đồng thời làm rõ ảnh hưởng của từng vấn đề đến chất lượng phần mềm.

Từ các kết quả đánh giá, báo cáo đề xuất giải pháp khắc phục và sắp xếp thứ tự ưu tiên cải tiến dựa trên mức độ ảnh hưởng, tính cần thiết và sự phụ thuộc giữa các công việc. Mỗi đề xuất cần gắn với tiêu chí hoàn thành và phương pháp kiểm chứng bằng kiểm thử hồi quy, nhằm xác nhận lỗi đã được xử lý, hạn chế phát sinh lỗi mới và từng bước nâng cao tính ổn định, khả năng bảo trì cũng như mức độ đáp ứng yêu cầu của hệ thống.

1.3. Phạm vi và phương pháp tổng hợp

Báo cáo tập trung kiểm định ứng dụng quản lý thư viện nhạc trực tuyến và các thành phần trực tiếp tham gia vào quá trình vận hành, xử lý nghiệp vụ và kiểm thử. Phạm vi đánh giá bao gồm mã Java, template HTML, CSS, cấu hình Maven, mã kiểm thử, collection Postman, kịch bản JMeter, bảng test case Excel và các báo cáo kết quả của dự án.

Phương pháp kiểm định kết hợp phân tích tĩnh mã nguồn với đối chiếu kết quả thực thi kiểm thử. Mã nguồn được đọc để xác định kiến trúc, luồng xử lý, quy tắc nghiệp vụ và cơ chế phân quyền. Kết quả JUnit, Postman, Selenium, JMeter và trạng thái trong bảng test case được tổng hợp theo từng nhóm chức năng, sau đó đối chiếu với hành vi được cài đặt để xác định điểm đạt, điểm không đạt, trường hợp bị cản trở và nội dung chưa có kết quả.

Nội dung đánh giá được trình bày theo ba nhóm. Nhóm thứ nhất mô tả hiện trạng kiến trúc, dữ liệu, nghiệp vụ và phân quyền xác định từ mã nguồn. Nhóm thứ hai trình bày toàn bộ kết quả kiểm thử thực tế được ghi nhận trong các báo cáo và bảng test case. Nhóm thứ ba gồm các vấn đề phát hiện, mức độ ảnh hưởng và đề xuất cải tiến chưa được triển khai trong mã nguồn.

Các số liệu trong báo cáo được lấy trực tiếp từ kết quả thực thi đã ghi nhận: báo cáo Surefire và tệp ketqua.txt cho JUnit, bảng trạng thái trong workbook test case, bảng kết quả Postman, tài liệu Selenium và kết quả JMeter. Các nguồn được kiểm tra chéo theo tổng số ca, số đạt, số không đạt, số bị bỏ qua hoặc bị cản trở. Báo cáo giữ nguyên kết quả đo được; các nhận xét và đề xuất được tách riêng để không làm thay đổi ý nghĩa của số liệu kiểm thử.

1.4. Bố cục

Các phần tiếp theo của báo cáo được sắp xếp theo trình tự từ tổng quan hệ thống đến kết quả đánh giá và định hướng hoàn thiện. Trước hết, báo cáo trình bày kiến trúc ứng dụng và tổ chức dữ liệu nhằm làm rõ các thành phần chính, mối quan hệ giữa các lớp xử lý và cách hệ thống lưu trữ thông tin. Tiếp theo là nội dung phân tích nghiệp vụ, các chức năng đã triển khai và cơ chế phân quyền đối với từng nhóm người dùng.

Trên cơ sở đó, báo cáo mô tả chiến lược kiểm thử, phạm vi áp dụng và phương pháp đánh giá chất lượng phần mềm. Kết quả kiểm thử JUnit được tổng hợp, sau đó đối chiếu với bảng test case để xác định mức độ đáp ứng yêu cầu và tính nhất quán giữa kịch bản với kết quả đã ghi nhận. Các nội dung kiểm thử endpoint, giao diện và hiệu năng được trình bày riêng để làm rõ chất lượng của từng khía cạnh trong hệ thống.

Cuối cùng, báo cáo tổng hợp những vấn đề đã phát hiện, phân tích mức độ ảnh hưởng và đề xuất kế hoạch cải tiến theo mức độ ưu tiên. Phần kết luận đưa ra nhận định chung về hiện trạng của ứng dụng, đồng thời danh mục minh chứng được cung cấp nhằm hỗ trợ việc tra cứu, kiểm chứng và đối chiếu các nội dung đã trình bày.

2. KIẾN TRÚC VÀ MÔ HÌNH DỮ LIỆU

2.1. Công nghệ sử dụng

Bảng 2.1. Công nghệ và vai trò trong hệ thống

Thành phần	Công nghệ và vai trò
Ngôn ngữ và nền tảng	Java 17 và Spring Boot 3.5.6 theo web/pom.xml
Xử lý web	Spring MVC tiếp nhận request và trả view hoặc redirect
Giao diện	Thymeleaf, Bootstrap 5.3.3 và CSS dùng chung
Lưu trữ	Spring Data JPA; MySQL driver cho runtime, H2 cho test
Xác thực	Spring Security, tài khoản trong DB, BCrypt
Kiểm thử	JUnit 5, Mockito, MockMvc, pytest, Selenium, Postman, JMeter

2.2. Luồng xử lý

Trình duyệt gửi request qua chuỗi bộ lọc Spring Security. Khi được phép truy cập, controller nhận dữ liệu form và gọi service. Service kiểm tra điều kiện nghiệp vụ rồi sử dụng repository để đọc hoặc ghi dữ liệu qua JPA. Controller đưa dữ liệu vào Model và trả tên template Thymeleaf; thao tác ghi thành công thường chuyển hướng về trang danh sách.

Luồng chính có thể biểu diễn như sau: Trình duyệt - Spring Security - Controller - Service - Repository - Cơ sở dữ liệu. Chiều trả về đi qua Model và template để tạo HTML. Đây là ứng dụng web render phía máy chủ; các endpoint trong collection Postman chủ yếu trả HTML hoặc redirect, không phải một bộ REST API JSON độc lập.

2.3. Các thực thể

Bảng 2.2. Các thực thể và thuộc tính nghiệp vụ chính

Thực thể	Thuộc tính nghiệp vụ chính
User	username, email, role, password ánh xạ cột password_hash
Song	title, artist, genre, releaseYear, audioFilePath, duration
Artist	name, country, gender, informations
Genre	name, informations

Các thực thể trên đều có id kiểu String sinh bằng UUID. Song lưu tên nghệ sĩ và thể loại dưới dạng chuỗi, chưa thiết lập quan hệ JPA với Artist và Genre. Do đó, đổi tên hoặc xóa nghệ sĩ, thể loại chưa tự cập nhật dữ liệu bài hát. Đây là hạn chế về tính nhất quán cần được xử lý nếu mở rộng hệ thống.

3. CHỨC NĂNG VÀ PHÂN QUYỀN

3.1. Tài khoản và phiên đăng nhập

AuthController cung cấp trang đăng nhập và đăng ký. Khi đăng ký, UserService yêu cầu username, email không trống, mật khẩu ít nhất 8 ký tự và kiểm tra trùng username hoặc email không phân biệt viết hoa hay thường. Vai trò được ép về User; mật khẩu được mã hóa trước khi lưu. DatabaseUserDetailsService đọc tài khoản từ repository và ánh xạ Admin thành ROLE_ADMIN, các trường hợp còn lại thành ROLE_USER.

Spring Security cho phép truy cập công khai trang chủ, /login, /register, CSS và ảnh. Các đường dẫn /users/** yêu cầu vai trò ADMIN; các đường dẫn còn lại yêu cầu đăng nhập. Đăng xuất hủy session và xóa cookie JSESSIONID. Cấu hình không tắt CSRF, các form POST dùng th:action để tích hợp token.

3.2. Các nhóm nghiệp vụ

Bảng 3.1. Chức năng và quyền truy cập theo nhóm nghiệp vụ

Nhóm	Chức năng đã triển khai	Quyền truy cập
Bài hát	Danh sách, tìm theo tên hoặc nghệ sĩ hoặc thể loại, thêm, sửa, xóa	Người dùng đã đăng nhập
Nghệ sĩ	Danh sách, tìm theo tên, thêm, sửa, xóa	Người dùng đã đăng nhập
Thể loại	Danh sách, tìm theo tên, thêm, sửa, xóa	Người dùng đã đăng nhập
Người dùng	Danh sách, tìm username hoặc email, thêm, sửa, xóa	Admin

Theo cấu hình hiện tại, người dùng thường có thể thay đổi dữ liệu bài hát, nghệ sĩ và thể loại. Báo cáo ghi nhận đây là hành vi đang được triển khai; cần đối chiếu yêu cầu phân quyền của đề tài trước khi quyết định có giới hạn thao tác ghi cho Admin hay không.

3.3. Giao diện

Các trang quản lý dùng bảng dữ liệu, ô tìm kiếm và modal Bootstrap cho thao tác thêm, sửa, xóa. Một số ràng buộc như required, minlength và type=email hoặc type=url được áp dụng trên trình duyệt. Việc kiểm tra phía máy chủ chưa đồng đều giữa thao tác tạo và cập nhật.

Trang bài hát hiển thị liên kết audioFilePath mở trong tab mới. Chưa có phần tử trình phát nhúng, chức năng Play/Pause hoặc chuyển bài. Mã nguồn cũng chưa có thực thể, service, controller hoặc giao diện cho playlist và danh sách yêu thích. Các chức năng này phải được ghi nhận là chưa triển khai, không đưa vào danh sách chức năng đã hoàn thành.

4. CHIẾN LƯỢC VÀ THIẾT KẾ KIỂM THỬ

4.1. Các cấp độ kiểm thử

Bảng 4.1. Các cấp độ kiểm thử và phạm vi áp dụng

Cấp độ	Cách tổ chức	Giới hạn cần lưu ý
Unit	Mockito giả lập repository và PasswordEncoder để kiểm tra service	Không chứng minh truy vấn DB hoặc mã hóa BCrypt thật
Controller	MockMvc standalone, service giả lập và GlobalException thật	Không nạp chuỗi Security và không render Thymeleaf thật
Tích hợp	CrudFlowTest dùng JPA, các service và H2	Không thay thế kiểm thử MySQL và toàn bộ ứng dụng
Endpoint	Postman gửi form, giữ cookie, lấy CSRF, kiểm tra status và redirect	Cần ứng dụng đang chạy và dữ liệu kiểm thử phù hợp
Giao diện	Selenium dùng Page Object Model, fixture và explicit wait	Cần Chrome và ứng dụng; một số ca cần Admin
Hiệu năng	JMeter đăng nhập rồi lặp hành trình đọc dữ liệu	Phụ thuộc cấu hình tải, môi trường và tính đúng của phiên đăng nhập

4.2. Kỹ thuật thiết kế ca kiểm thử

Phân vùng tương đương được thể hiện qua dữ liệu hợp lệ, thiếu trường bắt buộc, tên trùng và id không tồn tại. Giá trị biên được kiểm tra qua mật khẩu 7 và 8 ký tự. Các ca tìm kiếm bao gồm: không phân biệt viết hoa hay thường, từ khóa rỗng, ký tự đặc biệt và dữ liệu tiếng Việt.

Kiểm thử trạng thái được áp dụng cho người chưa đăng nhập, người dùng thường, Admin và phiên sau đăng xuất. Luồng tích hợp kiểm tra chuỗi tạo, đọc, sửa, tìm kiếm và xóa. Những kỹ thuật này giúp bao quát cả nhánh thành công và nhánh lỗi, nhưng chưa có báo cáo độ phủ mã để xác định phần trăm dòng hoặc nhánh đã được thực thi.

4.3. Truy vết và cách đọc kết quả

Bảng Excel sử dụng mã *TC_ART, TC_GEN, TC_SNG, TC_USR, TC_CTL, TC_OTH, TC_AUT, TC_PLY, TC_PLL, TC_FAV và TC_PRF*. Một số test Java có @DisplayName chứa mã TC; Selenium ghi mã trong tên hoặc mô tả test. Tuy nhiên, một test tự động có thể kiểm tra nhiều mã hoặc nhiều tình huống, nên số test chạy không bằng số dòng trong bảng test case.

Pass là đạt kỳ vọng của ca test cụ thể; Fail là đã thực hiện nhưng sai kỳ vọng; Blocked là chưa thể thực hiện do thiếu chức năng hoặc điều kiện; Chưa test là chưa có kết quả được ghi nhận. Skipped và xfail trong framework cần được giải thích riêng, không tự chuyển thành Pass.

5. BẢNG TEST CASE VÀ THỐNG KÊ

Việc đối chiếu được thực hiện dựa trên cột trạng thái của 11 sheet nghiệp vụ trong bảng test case TestCase_QuanLyThuVienNhac.xlsx. Ngoài các sheet này, workbook còn có một sheet "Tổng quan", nâng tổng số lên 12 sheet. Các test case được phân loại theo năm trạng thái gồm tổng số trường hợp, đạt (Pass), không đạt (Fail), bị cản trở (Blocked) và chưa kiểm thử (Chưa test).

Bảng 5.1. Tổng hợp trạng thái thực hiện test case theo nhóm chức năng

Nhóm	Tổng	Pass	Fail	Blocked	Chưa test
Artist Service	27	24	0	0	3
Genre Service	27	24	0	0	3
Song Service	25	24	0	0	1
User Service	21	20	0	0	1
Controller	39	32	6	0	1
Exception và khác	13	8	0	1	4
Đăng ký và đăng nhập	26	26	0	0	0
Phát nhạc và tìm kiếm	23	21	0	2	0
Playlist	17	0	0	17	0
Yêu thích	12	0	0	12	0
Hiệu năng JMeter	12	10	2	0	0
Tổng cộng	**242**	**190**	**8**	**32**	**12**

Nhóm "Controller" có 39 test case, gồm 32 trường hợp đạt, 6 trường hợp không đạt và 1 trường hợp chưa được kiểm thử. Đây là nhóm có số trường hợp thất bại nhiều nhất trong bảng test case, đồng thời tương ứng với các lỗi được ghi nhận tại những lớp kiểm thử controller trong phần kết quả JUnit. Vì vậy, cần ưu tiên kiểm tra các luồng tiếp nhận yêu cầu, ánh xạ dữ liệu, xác thực đầu vào, phản hồi lỗi và điều hướng giao diện tại tầng controller.

Đối với nhóm "Exception và khác", 8 trong tổng số 13 test case đã đạt, 1 trường hợp bị cản trở và 4 trường hợp chưa được kiểm thử. Mặc dù chưa ghi nhận trường hợp thất bại, các trạng thái chưa hoàn tất cho thấy cơ chế xử lý ngoại lệ và những chức năng liên quan vẫn cần được xác minh thêm trước khi đánh giá đầy đủ.

Các nhóm nghiệp vụ liên quan trực tiếp đến người dùng còn có mức độ hoàn thành thấp. Toàn bộ 26 test case của nhóm "Đăng ký và Đăng nhập" chưa được kiểm thử. Nhóm "Phát nhạc và tìm kiếm" có 2 trường hợp bị cản trở và 21 trường hợp chưa kiểm thử. Hai nhóm "Playlist" và "Yêu thích" có tổng cộng 29 test case nhưng đều đang ở trạng thái Blocked. Đây là khoảng trống kiểm thử đáng chú ý vì các nhóm này đại diện cho những chức năng cốt lõi của hệ thống thư viện nhạc trực tuyến.

Nhóm “Hiệu năng JMeter” có 12 test case, trong đó 3 trường hợp đạt, 2 trường hợp không đạt và 7 trường hợp chưa được kiểm thử. Kết quả này chưa đủ để kết luận toàn diện về khả năng đáp ứng và độ ổn định của hệ thống dưới tải. Các trường hợp thất bại cần được phân tích dựa trên tiêu chí hiệu năng đã đặt ra, đồng thời những kịch bản còn lại cần được thực hiện trong môi trường phù hợp.

Nhìn chung, bảng đối chiếu cho thấy các kiểm thử ở tầng dịch vụ đã đạt kết quả tốt, nhưng mức độ bao phủ giữa các nhóm chưa đồng đều. Tầng controller vẫn tồn tại lỗi, trong khi nhiều chức năng hướng tới người dùng và kịch bản hiệu năng chưa được thực hiện hoặc đang bị cản trở. Do đó, cần xử lý các điều kiện gây Blocked, bổ sung môi trường và dữ liệu kiểm thử, sau đó ưu tiên thực hiện những test case chưa hoàn tất trước khi đánh giá chất lượng tổng thể của hệ thống.

6. KIỂM THỬ MÃ NGUỒN BẰNG JUNIT

Test nằm tại app/MainProject/QuanLyThuVienNhacOnline/web/src/test/java/com/musicmanager/web/ :

Thư mục	Nội dung
service/	Unit test 4 service bằng Mockito: nhánh hợp lệ và không hợp lệ (tên rỗng, trùng tên, id không tồn tại, mật khẩu ngắn, mã hóa mật khẩu, register...)
controller/	MockMvc standalone cho 5 controller (TestMvc dựng sẵn, không nạp Spring context nên không phụ thuộc Security/DB)
exception/	GlobalException: ánh xạ 404, 400, 500 và tên view error/...
integration/	Luồng tạo → xem → sửa → tìm → xóa cho từng module trên H2 (@DataJpaTest, rollback sau mỗi test)

src/test/resources/application.properties dùng H2 trong bộ nhớ để test không phụ thuộc MySQL trên cloud và không làm bẩn dữ liệu thật.

Bảng dưới đây tổng hợp kết quả thực thi JUnit theo từng lớp kiểm thử từ tệp ketqua.txt và báo cáo Surefire. Các chỉ số gồm tổng số test, số test đạt (Pass), số test không đạt (Fail) và số test không được thực thi (Skipped):

Bảng 6.1. Tổng hợp kết quả kiểm thử JUnit theo lớp kiểm thử

Lớp test	Tổng	Pass	Fail	Skipped
ArtistServiceTest	23	23	0	0
GenreServiceTest	23	23	0	0
SongServiceTest	24	24	0	0
UserServiceTest	21	21	0	0
CrudFlowTest	4	4	0	0
GlobalExceptionTest	4	4	0	0
HomeControllerTest	1	1	0	0
ArtistControllerTest	10	9	1	0
GenreControllerTest	10	9	1	0
SongControllerTest	9	6	3	0
UserControllerTest	8	7	1	0
WebApplicationTests	1	0	0	1
Tổng cộng	**138**	**131**	**6**	**1**

Kết quả ghi nhận tổng cộng 138 trường hợp kiểm thử, trong đó có 131 trường hợp đạt, 6 trường hợp không đạt, 0 lỗi và 1 trường hợp bị bỏ qua. Tỷ lệ đạt là 94,93%, tỷ lệ không đạt là 4,35% và tỷ lệ bị bỏ qua là 0,72%.

Các lớp kiểm thử thuộc tầng dịch vụ, gồm ArtistServiceTest, GenreServiceTest, SongServiceTest và UserServiceTest, đạt toàn bộ trường hợp kiểm thử. CrudFlowTest, GlobalExceptionTest và HomeControllerTest cũng không ghi nhận trường hợp thất bại. Kết quả này cho thấy các nghiệp vụ đang được kiểm tra tại tầng service và các luồng tích hợp CRUD đều đáp ứng kỳ vọng của bộ test.

Sáu trường hợp không đạt tập trung ở các lớp kiểm thử controller. Cụ thể, ArtistControllerTest, GenreControllerTest và UserControllerTest mỗi lớp có 1 trường hợp thất bại; SongControllerTest có 3 trường hợp thất bại và là lớp có số lỗi nhiều nhất. Vì vậy, nhóm chức năng liên quan đến controller, đặc biệt là xử lý yêu cầu đối với bài hát, cần được ưu tiên kiểm tra nguyên nhân và xác minh lại trong lần kiểm thử tiếp theo.

Riêng lớp WebApplicationTests có 1 trường hợp bị bỏ qua và không có trường hợp nào được ghi nhận là đạt. Kết quả này chưa đủ để khẳng định kiểm thử khởi tạo ngữ cảnh ứng dụng đã thành công. Cần kiểm tra điều kiện bỏ qua, cấu hình môi trường và thực hiện lại trường hợp kiểm thử này khi hệ thống có đầy đủ điều kiện vận hành. Nhìn chung, kết quả cho thấy phần lớn trường hợp kiểm thử đã đạt, tuy nhiên các lỗi còn tồn tại và trường hợp bị bỏ qua vẫn cần được xử lý trước khi đưa ra kết luận cuối cùng về độ ổn định của ứng dụng.



Hình 6.1. Kết quả thực thi JUnit

6.1. Nguyên nhân sáu test thất bại

Ba test searchingMissingParam của ArtistControllerTest, GenreControllerTest và UserControllerTest mong HTTP 400 khi thiếu tham số bắt buộc, nhưng thực tế nhận 500. Ba test createInvalid, updateNotFound và deleteNotFound của SongControllerTest mong lần lượt 400, 404 và 404, nhưng đều nhận 500.

Nguyên nhân thể hiện trong mã nguồn là GlobalException có handler bắt Exception chung. Handler này tiếp nhận cả lỗi thiếu tham số và ResponseStatusException mà chưa có nhánh riêng bảo toàn mã trạng thái. Cách sửa đề xuất là ánh xạ rõ các loại lỗi này, đồng thời dùng trang lỗi phù hợp với từng nhóm HTTP status.

6.2. Ý nghĩa và giới hạn

Toàn bộ 91 test service và 4 luồng tích hợp đã pass. Kết quả này hỗ trợ đánh giá các tình huống đang được kiểm tra, nhưng không chứng minh mọi quy tắc nghiệp vụ đều đúng. Một số test còn chủ động ghi nhận hành vi hiện tại, chẳng hạn updateUser ghi đè null hoặc deleteUser không báo lỗi khi thiếu id.

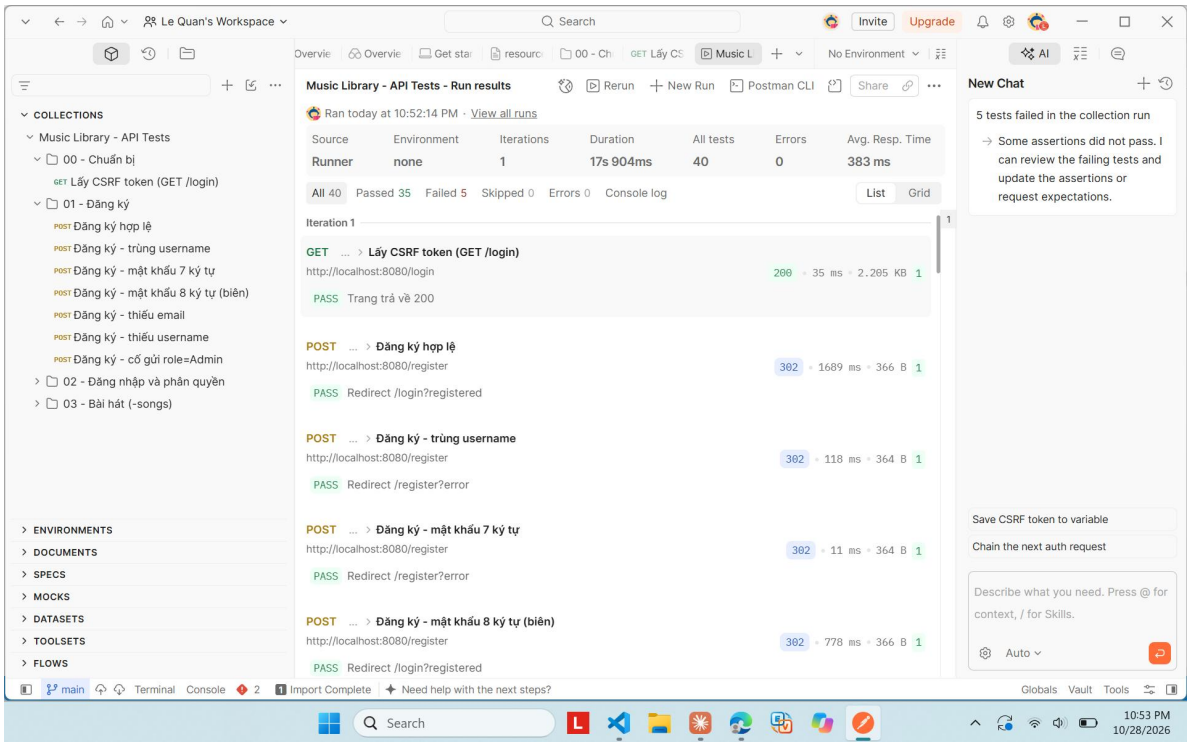
WebApplicationTests bị @Disabled. Các test controller dùng view giả và không chạy Security. Vì vậy, chưa thể dùng riêng kết quả JUnit để khẳng định toàn bộ giao diện render đúng, phân quyền hoạt động đúng trên server thật hoặc ứng dụng khởi động thành công trong môi trường mới.

7. KIỂM THỬ ENDPOINT BẰNG POSTMAN

7.1. Collection Postman

Collection JSON trong web/postman có 41 request, chia thành 7 thư mục từ 00 đến 06. Nội dung gồm lấy CSRF, đăng ký, đăng nhập và phân quyền, bài hát, nghệ sĩ, thể loại, đăng xuất. Một bản biểu diễn YAML nằm tại thư mục postman ở gốc repo.

Request chuẩn bị lấy token từ HTML. Sau đăng nhập, một request GET /songs lấy token mới cho phiên hiện tại. Các ca POST gửi dữ liệu form-urlencoded; assertion kiểm tra HTTP status và header Location. Collection có ca đăng ký mật khẩu 7 hoặc 8 ký tự, username trùng, cố gửi role=Admin, thiếu CSRF, truy cập /users bằng tài khoản thường và thao tác với id không tồn tại.



Hình 7.1. Kết quả thực thi Postman

Kết quả chạy Postman ghi nhận 40 test, trong đó có 35 test đạt và 5 test không đạt, tương ứng tỷ lệ đạt 87,50%. Lần chạy không có test bị bỏ qua hoặc lỗi thực thi, hoàn thành trong 17,904 giây và có thời gian phản hồi trung bình 383 ms. Collection hiện có 41 request; số 40 phản ánh phạm vi test của lần chạy được ghi nhận.

Bảng 7.1. Tổng hợp kết quả kiểm thử endpoint bằng Postman

Chỉ số	Kết quả
Request hoặc assertion đã thực hiện	40
Pass	35
Fail	5
Tỷ lệ Pass	87,50%

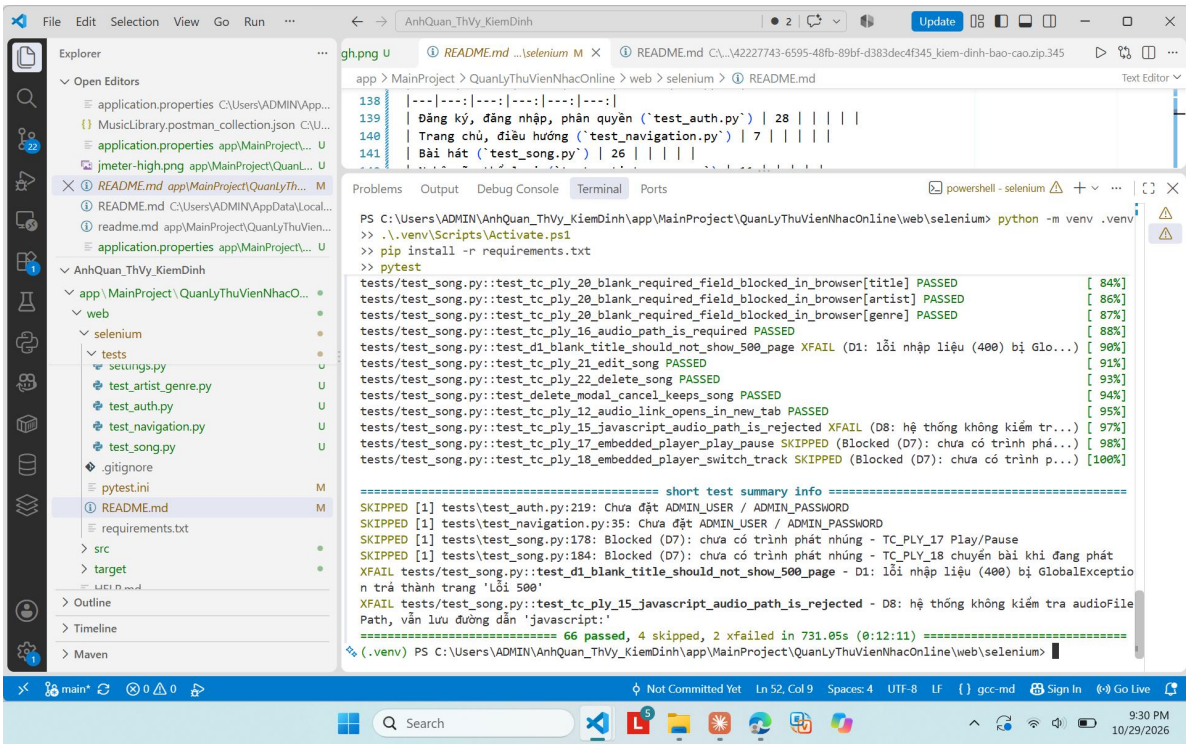
7.2. Bộ kiểm thử Selenium

Bảng 7.2. Số lượng ca kiểm thử Selenium theo nhóm chức năng

Nhóm kiểm thử Selenium	Số ca
Đăng ký đăng nhập và phân quyền	28
Trang chủ và điều hướng	7
Bài hát	26
Nghệ sĩ và thể loại	11
Tổng	**72**

Selenium hiện kiểm thử ứng dụng Music Library tại BASE_URL, mặc định là localhost:8080. Bộ test sử dụng Page Object Model cho Login, Register, Home, Songs, Artists, Genres và Users. Các trang CRUD kế thừa CrudPage; thao tác modal khoanh vùng theo .modal.show để giảm ảnh hưởng của các id bị lặp trong giao diện.

Fixture tạo trình duyệt riêng cho mỗi test, đăng ký tài khoản cho phiên chạy, sinh tên dữ liệu ngẫu nhiên và dọn các bản ghi CRUD đã tạo. Test lỗi được lưu ảnh màn hình. Tài khoản đăng ký không tự xóa vì người dùng thường không có quyền quản lý tài khoản. Hai ca Admin cần ADMIN_USER và ADMIN_PASSWORD; nếu thiếu thì bỏ qua.



Hình 7.2. Kết quả thực thi Selenium

Kết quả thực thi ghi nhận 66 test đạt, 4 test bị bỏ qua và 2 test xfail trong 731,05 giây. Bốn test bị bỏ qua gồm hai test cần tài khoản Admin và hai test liên quan đến trình phát nhúng chưa được triển khai. Hai test xfail phản ánh các lỗi đã biết: tiêu đề bài hát trống dẫn đến trang lỗi 500 và đường dẫn âm thanh dùng giao thức javascript chưa bị từ chối.

7.3. Đánh giá kết quả endpoint và giao diện

Kiểm thử Selenium được sử dụng để đánh giá các luồng thao tác thực tế trên giao diện, bao gồm đăng ký, đăng nhập, điều hướng, tìm kiếm và quản lý dữ liệu. Kết quả được xác định dựa trên URL sau thao tác, nội dung hiển thị, thông báo phản hồi và trạng thái của các thành phần trên trang.

Do Selenium không cung cấp trực tiếp mã trạng thái HTTP của từng phản hồi, các trường hợp liên quan đến mã 400, 403, 404 hoặc 500 được đối chiếu bằng Postman và MockMvc. Postman kiểm tra phản hồi của ứng dụng khi đang chạy, còn MockMvc xác nhận cách controller xử lý yêu cầu trong môi trường kiểm thử.

Việc kết hợp Selenium, Postman và MockMvc giúp đánh giá đồng thời giao diện người dùng, luồng nghiệp vụ và phản hồi HTTP. Một chức năng được xác nhận đạt khi thao tác trên giao diện cho kết quả đúng, quyền truy cập được áp dụng phù hợp và endpoint trả về mã trạng thái theo yêu cầu.

8. KIỂM THỬ HIỆU NĂNG BẰNG JMETER

8.1. Kịch bản và tiêu chí

Kịch bản music-library-load-test.jmx đăng nhập một lần cho mỗi người dùng ảo, giữ session bằng Cookie Manager và áp dụng thời gian nghỉ ngẫu nhiên 1-3 giây. Hành trình đọc gồm trang chủ, danh sách thể loại, tìm thể loại, danh sách nghệ sĩ, danh sách bài hát và tìm bài hát. Các tham số host, port, protocol, threads, rampup, duration và tài khoản được truyền từ thuộc tính JMeter.

Tài liệu định nghĩa ba mức tải 20, 50 và 100 người dùng trong 180 giây. Tiêu chí được nêu gồm tỷ lệ lỗi dưới 1%, thời gian trung bình dưới 1.000 ms và phân vị 95 dưới 2.000 ms. Đây là tiêu chí kiểm thử của dự án, chưa phải cam kết chất lượng đã đạt.

8.2. Kết quả kiểm thử với 120 người dùng

Kết quả kiểm thử với 120 luồng, ramp-up 1 giây và thời lượng cấu hình 15 giây ghi nhận 511 mẫu, thời gian phản hồi trung bình 3.078 ms, thông lượng khoảng 18,3 mẫu/giây và tỷ lệ lỗi 0,00%. Bảng 8.1 trình bày kết quả theo từng request.

Bảng 8.1. Kết quả kiểm thử JMeter với 120 người dùng

Request	Mẫu	Trung bình ms	Lỗi
GET login page	120	4	0,00%
POST login	120	282	0,00%
Trang chủ	120	4	0,00%
Danh sách thể loại	120	9291	0,00%
Tìm thể loại	31	13656	0,00%
Tổng	**511**	**3078**	**0,00%**

Thông lượng khoảng 18.3 mẫu/giây
Nhận xét:

- Không có request nào lỗi, nhưng danh sách và tìm kiếm thể loại rất chậm (trung bình 9.3 giây và 13.7 giây), vượt xa tiêu chí < 1 giây đã đặt ra. Trang chủ và đăng nhập nhanh.
- Trang chủ không truy vấn DB còn các trang chậm đều truy vấn DB, nên điểm nghẽn nhiều khả năng nằm ở truy vấn hoặc kết nối tới MySQL trên cloud (độ trễ mạng, số kết nối tối đa của pool mặc định) khi 120 luồng vào cùng lúc. Đây là giả thuyết, chưa kiểm chứng; cần đo thêm (ví dụ bật log thời gian truy vấn, tăng spring.datasource.hikari.maximum-pool-size, chạy lại).
- Lần chạy chỉ ghi nhận 31 mẫu tìm kiếm thể loại và chưa có mẫu cho nghệ sĩ, bài hát, tìm kiếm bài hát, nên chưa đủ để kết luận cho toàn bộ hành trình. Chưa có percentile 90/95.
- Lỗi 0.00% chỉ đúng với lần chạy này, không suy ra hệ thống chịu được mọi mức tải.

Việc cần làm tiếp theo: chạy lại theo ba mức tải 20/50/100 người dùng trong 180 giây, xuất báo cáo HTML để có percentile và bổ sung bảng kết quả.

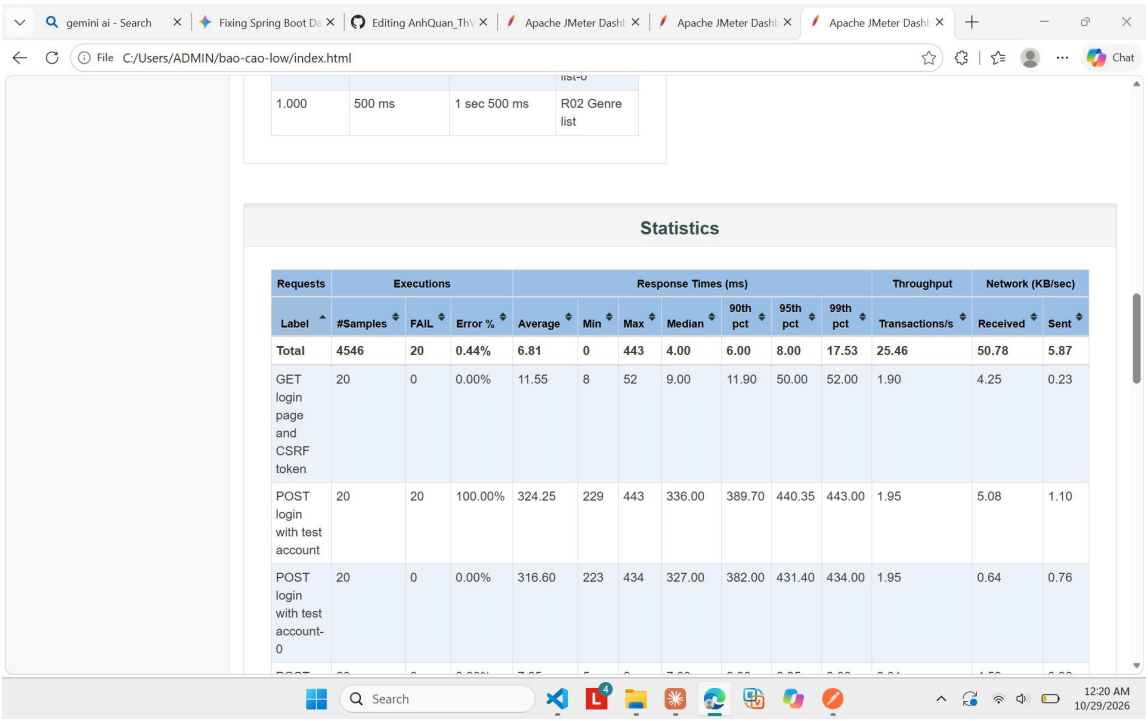
8.3. Kết quả theo ba mức tải

Kết quả theo ba mức tải cho thấy: mức 20 người dùng ghi nhận 4.546 mẫu, 20 mẫu lỗi, trung bình 6,81 ms, P95 đạt 8 ms và thông lượng 121,16 mẫu/giây; mức 50 người dùng ghi nhận 11.067 mẫu, 50 mẫu lỗi, trung bình 7,01 ms, P95 đạt 6 ms và thông lượng 62,43 mẫu/giây; mức 100 người dùng ghi nhận 21.563 mẫu, 100 mẫu lỗi, trung bình 5,78 ms, P95 đạt 6 ms và thông lượng 25,46 mẫu/giây. Tỷ lệ lỗi tổng thể tương ứng là 0,44%, 0,45% và 0,46%.

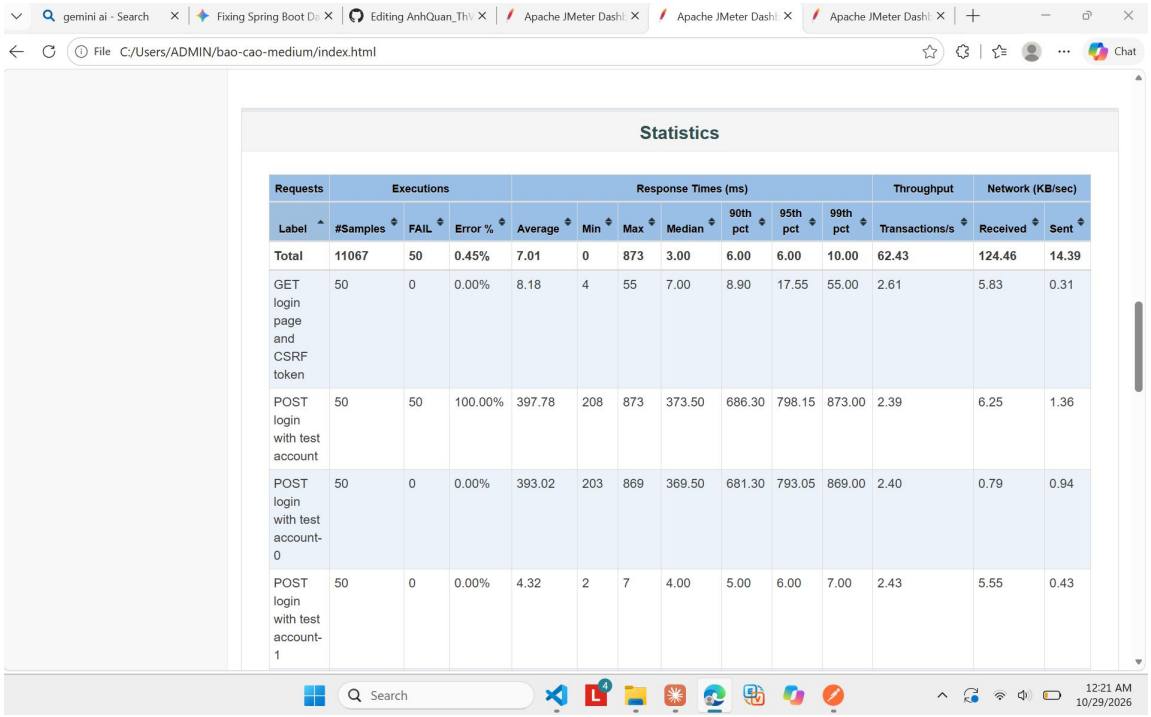
Bảng 8.2. Kết quả kiểm thử JMeter theo ba mức tải

Mức tải	Người dùng	Trung bình (ms)	95% (ms)	Lỗi (%)	Thông lượng (mẫu/giây)
Thấp	20	6,81	8	0,44	121,16
Trung bình	50	7,01	6	0,45	62,43
Cao	100	5,78	6	0,46	25,46

Dưới đây là hình ảnh kết quả theo 3 mức tải:



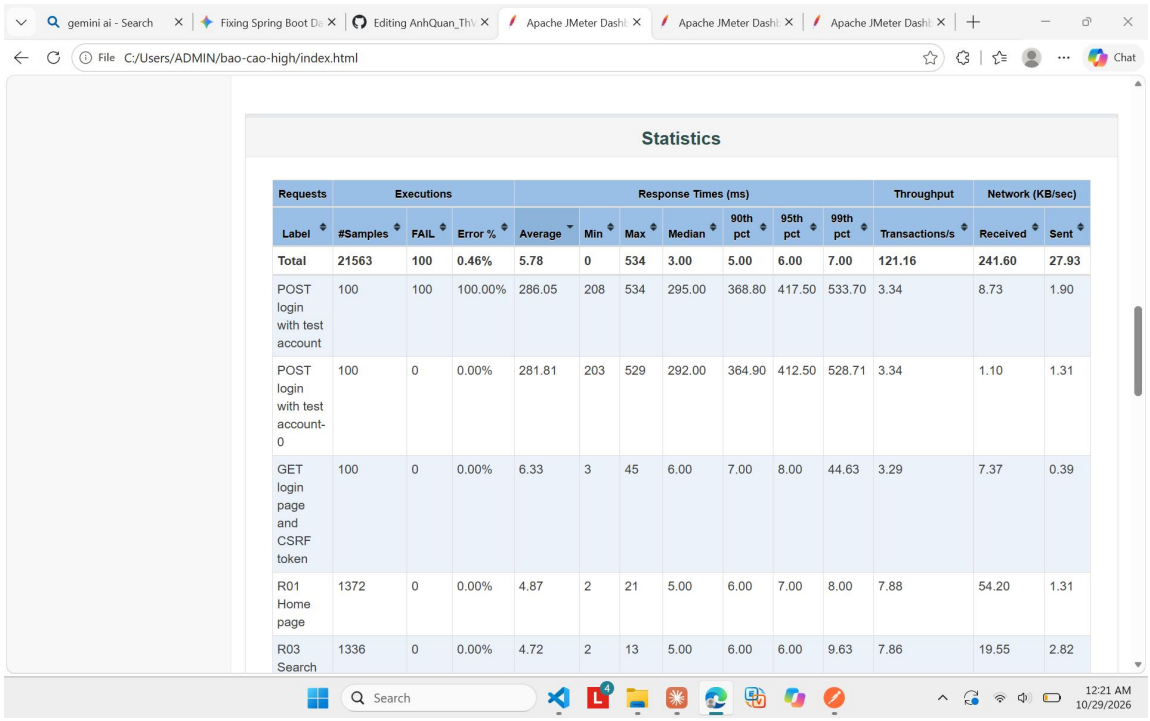
Hình 8.1. Kết quả JMeter ở mức tải 20 người dùng



Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**11067**	**50**	**0.45%**	**7.01**	**0**	**873**	**3.00**	**6.00**	**6.00**	**10.00**	**62.43**	**124.46**	**14.39**
GET login page and CSRF token	50	0	0.00%	8.18	4	55	7.00	8.90	17.55	55.00	2.61	5.83	0.31
POST login with test account	50	50	100.00%	397.78	208	873	373.50	686.30	798.15	873.00	2.39	6.25	1.36
POST login with test account-0	50	0	0.00%	393.02	203	869	369.50	681.30	793.05	869.00	2.40	0.79	0.94
POST login with test account-1	50	0	0.00%	4.32	2	7	4.00	5.00	6.00	7.00	2.43	5.55	0.43

Hình 8.2. Kết quả JMeter ở mức tải 50 người dùng



Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**21563**	**100**	**0.46%**	**5.78**	**0**	**534**	**3.00**	**5.00**	**6.00**	**7.00**	**121.16**	**241.60**	**27.93**
POST login with test account	100	100	100.00%	286.05	208	534	295.00	368.80	417.50	533.70	3.34	8.73	1.90
POST login with test account-0	100	0	0.00%	281.81	203	529	292.00	364.90	412.50	528.71	3.34	1.10	1.31
GET login page and CSRF token	100	0	0.00%	6.33	3	45	6.00	7.00	8.00	44.63	3.29	7.37	0.39
R01 Home page	1372	0	0.00%	4.87	2	21	5.00	6.00	7.00	8.00	7.88	54.20	1.31
R03 Search	1336	0	0.00%	4.72	2	13	5.00	6.00	6.00	9.63	7.86	19.55	2.82

Hình 8.3. Kết quả JMeter ở mức tải 100 người dùng

8.4. Đánh giá kết quả hiệu năng

Ba mức tải đều đáp ứng tiêu chí tổng thể về thời gian trung bình, P95 và tỷ lệ lỗi dưới 1%. Thông lượng tăng từ 25,46 lên 121,16 mẫu/giây khi số người dùng tăng từ 20 lên 100. Tuy nhiên, số mẫu lỗi ở mỗi mức tải bằng đúng số người dùng ảo và tập trung tại bước POST đăng nhập bằng tài khoản kiểm thử. Vì vậy, cần xác minh riêng điều kiện đăng nhập trước khi dùng các số liệu này để kết luận về toàn bộ hành trình nghiệp vụ.

9. CÁC VẤN ĐỀ PHÁT HIỆN TỪ MÃ NGUỒN

Các vấn đề trong phần này được tổng hợp từ kết quả kiểm thử thực tế và quá trình phân tích mã nguồn. Những lỗi đã xuất hiện trong quá trình kiểm thử được trình bày kèm theo minh chứng và kết quả liên quan. Các hạn chế khác được xác định thông qua việc rà soát cách triển khai, luồng xử lý và cấu trúc dữ liệu của hệ thống. Mức ưu tiên thể hiện thứ tự xử lý đề xuất dựa trên phạm vi ảnh hưởng và mức độ cần thiết.

Mã	Mô tả	Mức độ	Nguồn
D1	GlobalException có handler bắt Exception chung nên ResponseStatusException (Song) và thiếu tham số bắt buộc bị trả 500 thay vì 400/404. Làm fail 6 test: searchMissingParam (Artist, Genre, User), createInvalid, updateNotFound, deleteNotFound (Song). Cách sửa: thêm handler cho ResponseStatusException và MissingServletRequestParameterException.	Trung bình	Đã chạy test
D2	UserController xóa người dùng bằng GET /users/delete/{id}. GET không được bảo vệ bởi CSRF, nên một liên kết hoặc thẻ <img> trên trang khác có thể khiến quản trị viên xóa tài khoản vô ý. Nên dùng POST.	Cao	Rà soát code
D3	UserService.deleteUser không kiểm tra id tồn tại; updateUser ghi đè null lên dữ liệu cũ (khác Artist/Song).	Thấp	Rà soát code
D4	Song lưu nghệ sĩ và thể loại dạng chuỗi, không liên kết với bảng Artist/Genre, nên dữ liệu có thể không nhất quán (nghệ sĩ bị xóa vẫn còn trong bài hát).	Trung bình	Rà soát code
D5	Hiệu năng: danh sách và tìm kiếm thể loại trung bình 9.3 s và 13.7 s với 120 người dùng. Ngoài ra SongService.searchSongs tải toàn bộ bài hát rồi lọc trong bộ nhớ dù SongRepository đã có findBy...ContainingIgnoreCase, sẽ chậm khi dữ liệu lớn.	Cao	Đã chạy JMeter, rà soát code
D6	application.properties ghi cứng host và username DB trên cloud; mật khẩu để placeholder YOUR_PASSWORD nên dễ bị commit mật khẩu thật khi chạy. Nên dùng biến môi trường (ví dụ ${DB_PASSWORD}).	Trung bình	Rà soát code
D7	Hệ thống chưa có playlist, danh sách yêu thích và trình phát nhúng, dù là yêu cầu của đề tài. Các test case tương ứng ghi Blocked.	Cao (về yêu cầu)	Đối chiếu yêu cầu
D8	audioFilePath không được kiểm tra: có thể lưu javascript:... hoặc đường dẫn sai, và liên kết được hiển thị nguyên trong href (rủi ro XSS khi bấm).	Trung bình	Rà soát code
D9	Test cũ SongServiceTest sai so với code, và test @SpringBootTest phụ thuộc DB thật. Đã sửa: dùng H2,	Thấp	Đã chạy test

	viết lại test.		

9.1. Xử lý chưa chính xác mã trạng thái HTTP

Kết quả kiểm thử JUnit ghi nhận sáu trường hợp hệ thống trả sai mã trạng thái HTTP. Khi yêu cầu tìm kiếm nghệ sĩ không có tham số name, hệ thống trả mã 500 thay vì mã 400. Tương tự, khi yêu cầu xóa một bài hát không tồn tại, hệ thống trả mã 500 thay vì mã 404. Nguyên nhân là cơ chế xử lý ngoại lệ chung chưa phân biệt lỗi thiếu tham số, lỗi dữ liệu không tồn tại và lỗi nội bộ máy chủ. Ngoài ra, BadRequestException tuy trả đúng mã 400 nhưng lại sử dụng trang error/500, làm cho thông báo hiển thị không phù hợp với loại lỗi. Hệ thống cần xử lý riêng từng nhóm ngoại lệ và bảo đảm mã trạng thái HTTP thống nhất với nội dung trang lỗi.

9.2. Sử dụng phương thức GET để xóa tài khoản

Kết quả rà soát mã nguồn cho thấy UserController và form xác nhận đang sử dụng endpoint GET /users/delete/{id} để thực hiện thao tác xóa tài khoản. Cách triển khai này chưa phù hợp vì phương thức GET chỉ nên dùng để truy xuất dữ liệu, không nên làm thay đổi trạng thái của hệ thống. Ngoài ra, yêu cầu GET không được cơ chế CSRF của Spring Security bảo vệ như các yêu cầu POST, PUT, PATCH hoặc DELETE, làm tăng nguy cơ tài khoản bị xóa ngoài ý muốn khi người quản trị truy cập một đường dẫn được tạo sẵn.

Chức năng này nên được chuyển sang phương thức POST hoặc DELETE, đồng thời bổ sung token CSRF cho form xác nhận và cập nhật các test case liên quan. Sau khi sửa, cần kiểm tra yêu cầu hợp lệ có thể xóa tài khoản, yêu cầu thiếu token CSRF bị từ chối và endpoint GET cũ không còn thực hiện thao tác xóa dữ liệu.

9.3. Kiểm tra và liên kết dữ liệu chưa nhất quán

Kết quả rà soát mã nguồn cho thấy các quy tắc kiểm tra dữ liệu chưa được áp dụng thống nhất giữa thao tác tạo mới và cập nhật. Phương thức updateUser có thể ghi đè username, email và role ngay cả khi giá trị mới là null. Khi không tìm thấy mã tài khoản, phương thức này phát sinh RuntimeException chung thay vì trả về lỗi cụ thể. Phương thức deleteUser cũng chưa kiểm tra tài khoản có tồn tại trước khi xóa. Ngoài ra, createUser mới chỉ kiểm tra mật khẩu không được để trống, chưa áp dụng đầy đủ các quy tắc đang được sử dụng cho chức năng đăng ký. Các thao tác cập nhật nghệ sĩ, thể loại và bài hát cũng chưa kiểm tra thống nhất trường tên hoặc tiêu đề bị bỏ trống.

Hệ thống cần sử dụng chung một bộ quy tắc kiểm tra dữ liệu cho cả thao tác tạo mới và cập nhật. Các trường bắt buộc phải được kiểm tra trước khi lưu; giá trị null không được tự động ghi đè dữ liệu hiện có nếu người dùng không yêu cầu thay đổi. Trường hợp không tìm thấy dữ liệu cần trả về ngoại lệ và mã trạng thái phù hợp, đồng thời phải bổ sung test case cho dữ liệu trống, dữ liệu null, mã không tồn tại và dữ liệu trùng lặp.

Mô hình dữ liệu cũng chưa thiết lập quan hệ khóa ngoại giữa bài hát với nghệ sĩ và thể loại. Thông tin nghệ sĩ và thể loại hiện được lưu trực tiếp dưới dạng chuỗi trong bản ghi bài hát. Vì vậy, khi tên nghệ sĩ hoặc thể loại được thay đổi hoặc bản ghi liên quan bị xóa, thông tin trong bài hát có thể không được cập nhật, dẫn đến dữ liệu không nhất quán.

Trước khi thay đổi mô hình, cần xác định rõ cách xử lý các bài hát đang tham chiếu đến nghệ sĩ hoặc thể loại. Hệ thống có thể không cho phép xóa dữ liệu đang được sử dụng, yêu cầu

chuyển sang bản ghi khác hoặc áp dụng cơ chế cập nhật phù hợp. Sau khi thiết lập quan hệ, cần bổ sung kiểm thử tích hợp để xác nhận tính toàn vẹn dữ liệu khi thêm, sửa và xóa bản ghi.

9.4. Hạn chế trong tìm kiếm và kiểm tra đường dẫn nội dung

Kết quả rà soát mã nguồn cho thấy phương thức searchSongs gọi findAll() để lấy toàn bộ danh sách bài hát từ cơ sở dữ liệu, sau đó mới lọc kết quả trong bộ nhớ của ứng dụng. Cách xử lý này có thể làm tăng lượng dữ liệu phải truy xuất và thời gian xử lý khi số lượng bài hát lớn. Các trang danh sách hiện cũng chưa áp dụng phân trang, nên toàn bộ dữ liệu có thể được tải trong một lần truy cập.

Chức năng tìm kiếm nên sử dụng trực tiếp các phương thức truy vấn của repository để cơ sở dữ liệu thực hiện việc lọc. Đồng thời, các trang danh sách cần bổ sung phân trang và giới hạn số bản ghi trả về trong mỗi yêu cầu. Sau khi điều chỉnh, cần kiểm thử với nhiều quy mô dữ liệu để so sánh thời gian phản hồi, lượng dữ liệu truy xuất và mức sử dụng tài nguyên.

Việc kiểm tra các đường dẫn nội dung cũng chưa được thực hiện đầy đủ. Giá trị audioFilePath được đưa trực tiếp vào thuộc tính href, nhưng tầng service chưa kiểm tra giao thức hoặc định dạng của đường dẫn. Vì vậy, hệ thống có thể lưu những giá trị không hợp lệ hoặc các giao thức không được phép, chẳng hạn đường dẫn bắt đầu bằng javascript:. Các trường informations của nghệ sĩ và thể loại cũng cần được kiểm tra theo nguyên tắc tương tự nếu nội dung được sử dụng để tạo liên kết.

Hệ thống nên quy định rõ những loại đường dẫn được chấp nhận, chẳng hạn đường dẫn tương đối hoặc URL sử dụng giao thức HTTP và HTTPS, đồng thời từ chối các giao thức không an toàn và dữ liệu sai định dạng. Cần bổ sung test case cho đường dẫn hợp lệ, đường dẫn rỗng, URL sai định dạng và giao thức không được phép. Kết quả hiện tại chỉ cho thấy hệ thống chưa kiểm tra đầy đủ dữ liệu đầu vào; chưa có bằng chứng xác nhận mã JavaScript đã được thực thi thành công trên trình duyệt.

10. KHẢ NĂNG TÁI LẬP VÀ KẾ HOẠCH CẢI TIẾN

10.1. Cấu hình, quy trình build và tài liệu

Ứng dụng sử dụng MySQL trong môi trường vận hành và H2 trong môi trường kiểm thử. Các tệp application.properties được đặt trong thư mục tài nguyên tương ứng, giúp tách biệt cấu hình chạy ứng dụng và cấu hình kiểm thử. Thông tin kết nối cơ sở dữ liệu được truyền qua biến môi trường nhằm hạn chế việc lưu trực tiếp dữ liệu nhạy cảm trong mã nguồn.

Thư mục web được xác định là điểm bắt đầu chính thức để build và chạy ứng dụng. Maven Wrapper gồm mvnw, mvnw.cmd và các tệp cấu hình trong .mvn/wrapper, cho phép thực hiện quá trình build thống nhất mà không phụ thuộc vào Maven được cài đặt sẵn trên máy. Tệp pom.xml trong thư mục web quản lý đầy đủ các dependency cần thiết cho ứng dụng và bộ kiểm thử.

Tài liệu README trình bày cấu trúc dự án, yêu cầu môi trường, cách cấu hình cơ sở dữ liệu và các bước build ứng dụng. Các đường dẫn đến bảng test case, collection Postman, bộ kiểm thử Selenium và kịch bản JMeter được ghi theo đúng vị trí thực tế. Nội dung hướng dẫn cũng nêu rõ thứ tự thực hiện từng bộ kiểm thử và cách đọc kết quả, giúp quy trình kiểm định có thể được thực hiện lại trên một môi trường khác.

10.2. Kế hoạch cải tiến theo mức độ ưu tiên

Bảng 10.1. Kế hoạch cải tiến và tiêu chí hoàn thành

Ưu tiên	Công việc	Tiêu chí hoàn thành
P1	Hoàn thiện cơ chế xử lý ngoại lệ và mã trạng thái HTTP	Sáu test JUnit liên quan đến mã 400 và 404 đều đạt; mã trạng thái thống nhất với nội dung trang lỗi
P1	Chuyển chức năng xóa tài khoản từ GET sang POST hoặc DELETE	Yêu cầu GET không còn xóa được tài khoản; yêu cầu hợp lệ thực hiện thành công; yêu cầu thiếu token CSRF bị từ chối
P1	Thống nhất quy tắc kiểm tra dữ liệu đầu vào	Thao tác tạo và cập nhật đều từ chối dữ liệu trống, null, trùng lặp hoặc sai định dạng theo cùng một quy tắc
P1	Kiểm tra và giới hạn đường dẫn nội dung	Chỉ chấp nhận đường dẫn tương đối hoặc URL dùng giao thức được cho phép; các giao thức không an toàn bị từ chối
P2	Hoàn thiện quan hệ giữa bài hát, nghệ sĩ và thể loại	Dữ liệu vẫn nhất quán khi thêm, cập nhật hoặc xóa nghệ sĩ và thể loại; các test tích hợp liên quan đều đạt
P2	Tối ưu chức năng tìm kiếm và danh sách dữ liệu	Truy vấn tìm kiếm được thực hiện tại cơ sở dữ liệu; các trang danh sách có phân trang và giới hạn số bản ghi

P2	Hoàn thiện các chức năng playlist, yêu thích và trình phát nhạc	Các chức năng có đầy đủ giao diện, xử lý nghiệp vụ, lưu trữ dữ liệu và test case tương ứng
P2	Thực hiện kiểm thử hồi quy và đồng bộ kết quả	JUnit, Postman, Selenium và JMeter được thực hiện trên cùng một phiên bản; số liệu trong báo cáo và bảng test case thống nhất
P3	Đo độ bao phủ và bổ sung quy trình tích hợp liên tục	Báo cáo độ bao phủ mã nguồn và kết quả kiểm thử tự động được tạo, lưu lại sau mỗi lần cập nhật

10.3. Quy trình kiểm thử xác nhận sau cải tiến

Sau khi hoàn thành từng nội dung cải tiến, cần thực hiện kiểm thử hồi quy để xác nhận lỗi đã được khắc phục và không làm ảnh hưởng đến các chức năng đang hoạt động. Trước hết, chạy lại toàn bộ unit test, controller test và kiểm thử tích hợp trên cơ sở dữ liệu dành riêng cho kiểm thử. Kết quả phải được đối chiếu với các test case liên quan, đặc biệt là những trường hợp đã không đạt trước khi sửa.

Tiếp theo, sử dụng Postman để kiểm tra mã trạng thái HTTP, chuyển hướng, token CSRF, phiên đăng nhập và quyền truy cập của từng nhóm người dùng. Sau khi các endpoint hoạt động đúng, chạy Selenium để kiểm tra toàn bộ luồng thao tác trên giao diện, gồm đăng ký, đăng nhập, tìm kiếm, thêm, cập nhật, xóa và phân quyền.

Kiểm thử hiệu năng bằng JMeter được thực hiện sau khi các luồng chức năng đã hoạt động ổn định. Mỗi lần đo cần lưu thông tin về cấu hình máy, cơ sở dữ liệu, quy mô dữ liệu, phiên bản mã nguồn, số người dùng đồng thời, thời gian tăng tải và thời lượng kiểm thử. Các tệp kết quả JTL, báo cáo HTML và số liệu về thời gian phản hồi, phân vị, thông lượng và tỷ lệ lỗi phải được lưu kèm báo cáo.

Quy trình xác nhận được xem là hoàn tất khi các test case ưu tiên đều đạt, không phát sinh lỗi nghiêm trọng mới, các bộ kiểm thử cho kết quả thống nhất và toàn bộ minh chứng được cập nhật theo cùng một phiên bản ứng dụng.

11. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Qua kiểm thử, hệ thống chạy ổn định về mặt chức năng cho các thao tác cơ bản (không có lỗi ở 131/138 test JUnit; không lỗi trong lần chạy JMeter), nhưng còn ba nhóm vấn đề cần xử lý: (1) xử lý lỗi trả sai mã trạng thái (D1) - cả 6 test fail đều từ nguyên nhân này; (2) hiệu năng các trang thể loại rất chậm khi nhiều người dùng truy cập đồng thời (D5); (3) thiếu playlist, yêu thích và trình phát nhúng so với yêu cầu đề tài (D7).

Hướng phát triển:

1. Sửa D1 rồi chạy lại toàn bộ test (kiểm thử hồi quy); các test fail sẽ chuyển sang Pass.
2. Hoàn thiện playlist, yêu thích và trình phát nhúng, rồi kích hoạt các test case đang Blocked.
3. Phân tích và tối ưu truy vấn thể loại; chạy lại JMeter ở 3 mức tải và xuất báo cáo có percentile.
4. Đổi xóa người dùng sang POST, kiểm tra audioFilePath, đưa thông tin DB ra biến môi trường.
5. Đo độ phủ mã bằng JaCoCo; chạy test tự động trên GitHub Actions

TÀI LIỆU THAM KHẢO

[1] Spring, “Spring Boot Reference Documentation,” https://docs.spring.io/spring-boot/index.html

[2] Spring, “Spring Security Reference,” https://docs.spring.io/spring-security/reference/

[3] Oracle, “Java Documentation,” https://docs.oracle.com/en/java/

[4] JUnit Team, “JUnit 5 User Guide,” https://junit.org/junit5/docs/current/user-guide/

[5] Mockito, “Mockito Framework Documentation,” https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html,

[6] Selenium, “Selenium Documentation,” https://www.selenium.dev/documentation/

[7] Postman, “Postman Documentation,” https://learning.postman.com/docs/

[8] Apache Software Foundation, “Apache JMeter User Manual,” https://jmeter.apache.org/usermanual/

[9] Apache Software Foundation, “Apache Maven Documentation,” https://maven.apache.org/guides/

[10] Thymeleaf, “Thymeleaf Documentation,” https://www.thymeleaf.org/documentation.html

[11] MySQL, “MySQL Reference Manual,” https://dev.mysql.com/doc/

[12] OWASP Foundation, “OWASP Web Security Testing Guide,” https://owasp.org/www-project-web-security-testing-guide/